
Department of CSE (Data Science)

II Year II Semester
DATA ENGINEERING LAB (R23DS22L2)

Roll Number: _____

Name of the Student: _____

Academic Year: 2024-2025



Narasaraopeta Engineering College

Kotappakonda Road, Yellamanda (Post), Narasaraopet – 522601, Guntur District, AP

Approved by AICTE, New Delhi & Permanently affiliated to JNTUK, Kakinada, Code: 47,

Accredited by NBA & NAAC, RTA Approved Pollution test Centre, ISO 9001: 2008 Certified

Institution. Phone: 08647-239905, Website: www.nrtec.in.



Approved by AICTE, Permanently affiliated to JNTUK, Accredited by NBA & NAAC with 'A+' Grade,
Kotappakonda Road, Yellamanda (V), Narasaraopeta, Andhra Pradesh – 522601, www.nrtec.in

DEPARTMENT OF
CSE (DATA SCIENCE)

CERTIFICATE

ACADEMIC YEAR: 2024-25

This is to certify that the bonafide record work done by _____

_____ bearing **Roll Number** _____

of **II B.Tech. II Semester** in the **Data Engineering Laboratory** is satisfactorily
completed.

Staff In-Charge

Head of the Department

Submitted for the practical examination held on _____

External Examiner

INDEX

S .no	Name of the Experiment	Page No	Date	Remarks
1	Write a program for Reading and writing files in python	1-10	22-02-2025	
2	Installing and configuring Apache Nifi, Apache Airflow	11-17	8-03-2025	
3	Working with Databases	18-25	15-03-2025	
4	Cleaning, Transforming and Enriching Data	26-27	22-03-2025	
5	Installing and configuring Elastic search, kibana, post gesture SQL	28-29	29-03-2025	
6	Building the Data Pipeline	30-40	5-04-2025	
7	Building a Kibana Dash Board	41-43	12-04-2025	
8	Perform the following operations a.staging and validating data	43-45	19-04-2025	
9	Version control with the NiFi Registry	46-48	26-04-2025	
10	Monitoring Data pipelines	49-51	3-05-2025	
11	Deploying Data Pipelines	52-59	8-05-2025	
12	Building a production Data Pipeline	60-68	17-05-2025	

WEEK 1

AIM : Write a Program for Reading and Writing Files in Python?

Input.txt

Hello, How are you!!

Output.txt

Hello, How are you!!

Processing.py

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime
import os

# Paths to input and output files
INPUT_FILE = '/config/workspace/question/dags/input.txt' # Change this to your actual input file
path
OUTPUT_FILE = '/config/workspace/question/dags/output.txt' # Change this to your actual output
file path

def read_and_write_file():
    if not os.path.exists(INPUT_FILE):
        raise FileNotFoundError(f"{INPUT_FILE} not found!")

    with open(INPUT_FILE, 'r') as infile:
        content = infile.read()

    with open(OUTPUT_FILE, 'w') as outfile:
        outfile.write(content)

    print(f"Successfully copied contents from {INPUT_FILE} to {OUTPUT_FILE}")

# Define the DAG
```

```

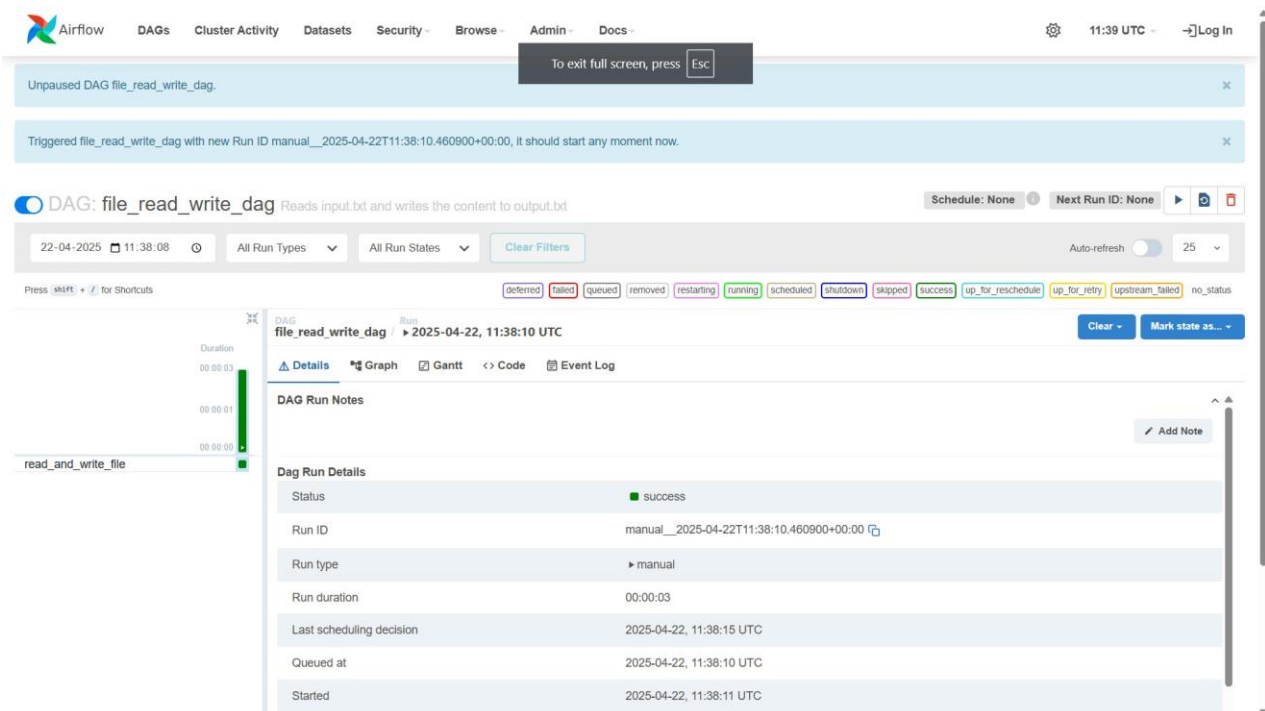
with DAG(
    dag_id='file_read_write_dag',
    description='Reads input.txt and writes the content to output.txt',
    schedule_interval=None, # Run manually
    start_date=datetime(2025, 4, 1),
    catchup=False,
    tags=['file-processing'],
) as dag:

    file_processing_task = PythonOperator(
        task_id='read_and_write_file',
        python_callable=read_and_write_file,
    )

    file_processing_task

```

Output:



AIM : Write a Program for Processing files in Airflow?

Simple_file_dag.py

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime
import os

# Folder where files will be placed manually
WATCH_FOLDER = '/tmp/input_files'

def process_files():
    if not os.path.exists(WATCH_FOLDER):
        print(f"{WATCH_FOLDER} does not exist. Creating it.")
        os.makedirs(WATCH_FOLDER)
        return

    files = os.listdir(WATCH_FOLDER)
    if not files:
        print("No files found.")
        return

    for file in files:
        path = os.path.join(WATCH_FOLDER, file)
        if os.path.isfile(path):
            with open(path, 'r') as f:
                content = f.read()
                print(f"--- {file} ---")
                print(content)
                print("----- ")
            # Optional: remove file after processing
            os.remove(path)

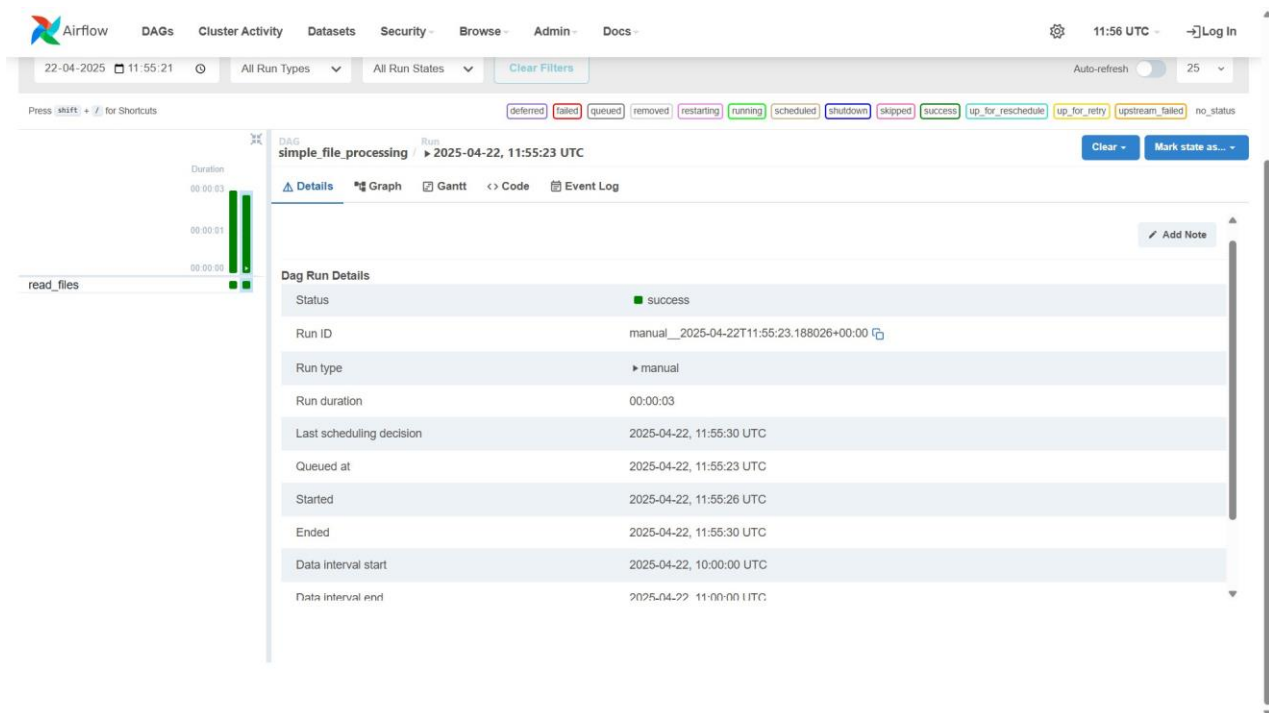
# Define the DAG
with DAG(
    dag_id='simple_file_processing',
    start_date=datetime(2024, 1, 1),
    schedule_interval='@hourly',
    catchup=False,
    description='A simple DAG to read and log text files from a folder',
```

) as dag:

```
read_task = PythonOperator(  
    task_id='read_files',  
    python_callable=process_files,  
)
```

read_task

Output:



AIM: Write a Program for NiFi processors for handling files?

Process:

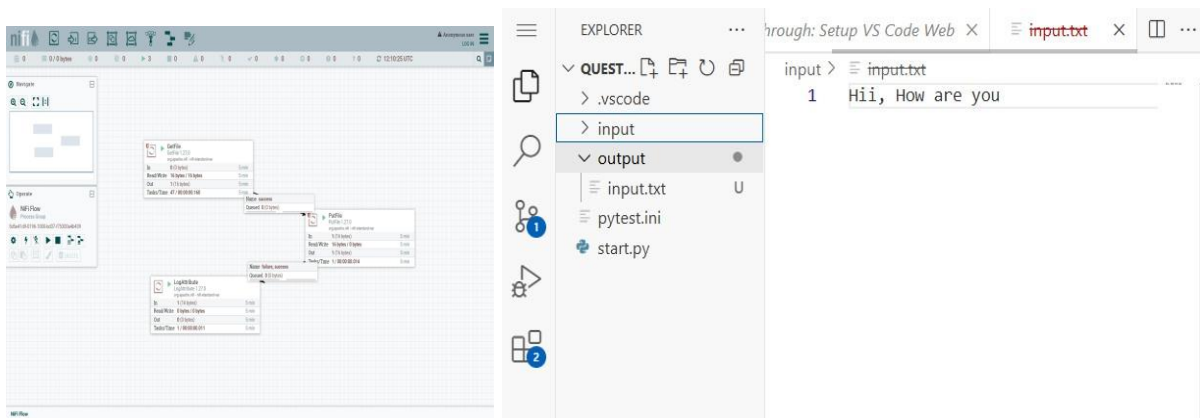
Processor	Purpose
GetFile	Ingest files from a source folder
LogAttribute	Debug/log file metadata
UpdateAttribute	Add custom metadata (optional)
ExecuteScript	Process content using code (Groovy, Python, etc.)
PutFile	Save the processed file

Input:

Input.txt:

Hi! How are you !!

Output:



AIM : Write a Program for Reading and writing data to databases in Python?

Action	Processor
Create test data	GenerateFlowFile
Insert into DB	PutDatabaseRecord
Read from DB	ExecuteSQL
View result	LogAttribute

Code :

mysql.py

```
import sqlite3

# Create an in-memory database (disappears when script ends)
conn = sqlite3.connect(':memory:')
cursor = conn.cursor()

# Create a table
cursor.execute("""
    CREATE TABLE students (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        age INTEGER
    )
""")

# Insert some records
students = [('Alice', 21), ('Bob', 22), ('Charlie', 23)]
cursor.executemany("INSERT INTO students (name, age) VALUES (?, ?)", students)

# Read and print all records
cursor.execute("SELECT * FROM students")
for row in cursor.fetchall():
    print(row)

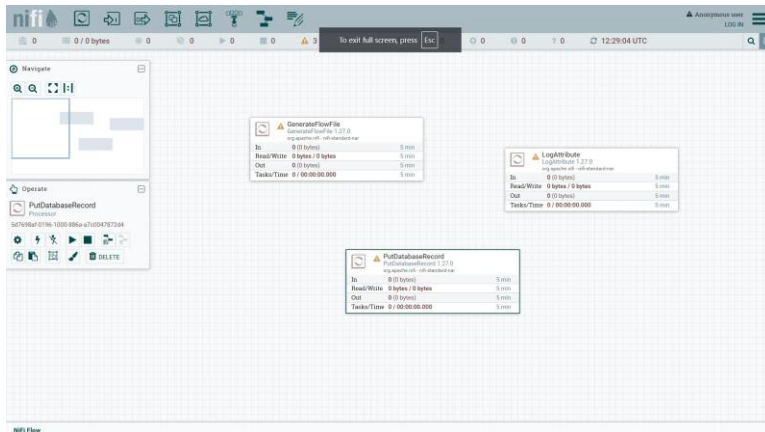
# Done
conn.close()
```

Output:

```

/bin/python3 /config/workspace/question/mysql.py
root@a62adc29f-b1f0-414d-b59b-c5bfb4fb8d9c-0:/config/workspace/question# /bin/python3 /config/workspace/question/mysql.py
(1, 'Alice', 21)
(2, 'Bob', 22)
(3, 'Charlie', 23)
root@a62adc29f-b1f0-414d-b59b-c5bfb4fb8d9c-0:/config/workspace/question#

```



AIM :

Code for Managing Databases in Apache Airflow

mysql_connection_dag.py

```

from airflow import DAG
from airflow.providers.mysql.operators.mysql import MySQLOperator
from datetime import datetime

# DAG definition
with DAG(
    dag_id='mysql_connection_dag',
    start_date=datetime(2025, 4, 22),
    schedule_interval=None,
    catchup=False,
    description='Example DAG to interact with MySQL using MySQLOperator',
    default_args={'owner': 'airflow'},
) as dag:

    # Create table task
    create_table = MySQLOperator(
        task_id='create_employee_table',

```

```

mysql_conn_id='my_mysql_connection', # this should match your connection ID in Airflow
sql="""
    CREATE TABLE IF NOT EXISTS employees (
        id INT AUTO_INCREMENT PRIMARY KEY,
        name VARCHAR(100),
        position VARCHAR(100)
    );
    """
    autocommit=True,
)

# Insert data task
insert_data = MySqlOperator(
    task_id='insert_employee_data',
    mysql_conn_id='my_mysql_connection',
    sql="""
        INSERT INTO employees (name, position)
        VALUES
            ('John Doe', 'Software Engineer'),
            ('Jane Smith', 'Data Scientist');
    """
    autocommit=True,
)

# Query data task
query_data = MySqlOperator(
    task_id='query_employee_data',
    mysql_conn_id='my_mysql_connection',
    sql="SELECT * FROM employees;",
    autocommit=True,
)

# Set task order
create_table >> insert_data >> query_data

```

Output:

```
mysql> SELECT * FROM employees;
```

```

+----+-----+-----+
| id | name   | position   |
+----+-----+-----+
| 1 | John Doe | Software Engineer |
| 2 | Jane Smith | Data Scientist   |

```

+---+-----+-----+

AIM : Write a Code for Database Processors in Apache NiFi

Code:

```
[GenerateTableFetch]
  ↓ [ExecuteSQL]
  ↓
[ConvertRecord (optional)]
  ↓
[PutFile / LogAttribute / PutDatabaseRecord]
```

1. Controller Service: DBCPConnectionPool

Name: MySQLConnectionPool

Type: DBCPConnectionPool

Properties:

- Database Connection URL: jdbc:mysql://localhost:3306/mydb
- DB Driver Class: com.mysql.cj.jdbc.Driver
- DB Driver Location: /usr/share/java/mysql-connector-java-8.0.xx.jar
- DB User: root
- DB Password: @123root

2. Processor: ExecuteSQL

Name: ExecuteSQL

Type: ExecuteSQL

Properties:

- Database Connection Pooling Service: MySQLConnectionPool
- SQL select query: SELECT * FROM employees;
- Output Format: Avro

3. Processor: ConvertRecord

Name: ConvertRecord

Type: ConvertRecord

Properties:

- Record Reader: AvroReader
- Record Writer: JsonRecordSetWriter

4. Processor: LogAttribute

Name: LogAttribute

Type: LogAttribute

Purpose: Logs the output from the SQL query in JSON format

Flow Connections

ExecuteSQL --> ConvertRecord --> LogAttribute

Output:

```
[
  {
    "id": 1,
    "name": "John Doe",
    "position": "Software Engineer"
  },
  {
    "id": 2,
    "name": "Jane Smith",
    "position": "Data Scientist"
  }
]
```

Week -2 (Working with Databases)

AIM : Write a code for Inserting and extracting relational data in Python?

Code:

```
# Step 1: Install PyMySQL (Run only once)
!pip install pymysql
# Step 2: Import necessary Libraries
import pymysql
import pandas as pd
# Step 3: Connect to MySQL Server
connection = pymysql.connect(
    host='localhost',
    user='your_username',      # ← Replace with your MySQL username
    password='your_password', # ← Replace with your MySQL password
    cursorclass=pymysql.cursors.DictCursor
)
print("✅ Connected to MySQL Server")
# Step 4: Create Database and Table
with connection.cursor() as cursor:
    cursor.execute("CREATE DATABASE IF NOT EXISTS employee;")
    cursor.execute("USE employee;")

    create_table = """
    CREATE TABLE IF NOT EXISTS employee (
        EmployeeID INT PRIMARY KEY,
        Ename VARCHAR(50),
        DeptID INT,
        Salary INT,
        Dname VARCHAR(50),
        Dlocation VARCHAR(50),
        JoiningDate DATE
    );
    """

    cursor.execute(create_table)
    connection.commit()
    print("✅ Database and Table ready")
# Step 5: Insert Multiple Records
insert_sql = """
INSERT INTO employee (EmployeeID, Ename, DeptID, Salary, Dname, Dlocation, JoiningDate)
VALUES (%s, %s, %s, %s, %s, %s, %s);
"""
```

```

data = [
    (1001, 'Alice', 1, 5000, 'HR', 'Mumbai', '2023-01-10'),
    (1002, 'Bob', 2, 6000, 'Engineering', 'Bangalore', '2022-09-01'),
    (1003, 'Charlie', 1, 5500, 'HR', 'Mumbai', '2023-02-12'),
    (1004, 'David', 3, 7000, 'Marketing', 'Delhi', '2021-07-15'),
    (1005, 'Eve', 2, 6200, 'Engineering', 'Bangalore', '2022-11-20')
]

with connection.cursor() as cursor:
    cursor.executemany(insert_sql, data)
    connection.commit()
    print("✅ Inserted multiple employee records")
# Step 6: Fetch All Employees
with connection.cursor() as cursor:
    cursor.execute("SELECT * FROM employee;")
    result = cursor.fetchall()
    df = pd.DataFrame(result)
    print("✅ All Employees")
    display(df)
# Step 7: Filter Employees from HR Department
with connection.cursor() as cursor:
    cursor.execute("SELECT * FROM employee WHERE Dname = 'HR';")
    hr_employees = cursor.fetchall()
    df_hr = pd.DataFrame(hr_employees)
    print("✅ HR Department Employees")
    display(df_hr)
# Step 8: Close Connection
connection.close()
print("🔒 Connection closed")

```

Output:

All Employees:

EmployeeID	Ename	DeptID	Salary	Dname	Dlocation	JoiningDate
1001	Alice	1	5000	HR	Mumbai	2023-01-10
1002	Bob	2	6000	Engineering	Bangalore	2022-09-01
1003	Charlie	1	5500	HR	Mumbai	2023-02-12
1004	David	3	7000	Marketing	Delhi	2021-07-15
1005	Eve	2	6200	Engineering	Bangalore	2022-11-20

HR Department Employees:

EmployeeID	Ename	DeptID	Salary	Dname	Dlocation	JoiningDate
1001	Alice	1	5000	HR	Mumbai	2023-01-10
1003	Charlie	1	5500	HR	Mumbai	2023-02-12

AIM:

Write a code for Inserting and extracting NoSQL database data in Python?

Code:

```
!pip install pymongo
from pymongo import MongoClient

# Connect to MongoDB
client = MongoClient("mongodb://localhost:27017/") # Replace with your MongoDB URI if needed

# Choose or create a database
db = client["test_db"]

# Choose or create a collection
collection = db["test_collection"]

# Inserting data
data = {
    "name": "Sushma",
    "age": 25,
    "profession": "Software Developer"
}
insert_result = collection.insert_one(data)
print(f"Data inserted with ID: {insert_result.inserted_id}")

# Extracting data
query = {"name": "Sushma"}
retrieved_data = collection.find(query)

# Display retrieved data
for document in retrieved_data:
    print(document)
```

Output:

Data inserted with ID: 60d5b0f7e9bdafeb520019d

```
{'_id': ObjectId('60d5b0f7e9bdafeb520019d'), 'name': 'Sushma', 'age': 25, 'profession': 'Software Developer'}
```


AIM :

Write a code for Building database pipelines in Airflow?

Code:

```
# airflow/dags/etl_pipeline.py

from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime
import pandas as pd
import psycopg2

# --- ETL Task Functions ---
def extract():
    df = pd.read_csv('/path/to/your/data.csv') # Replace with your actual path
    df.to_csv('/tmp/extracted_data.csv', index=False)
    print("✔ Data extracted")

def transform():
    df = pd.read_csv('/tmp/extracted_data.csv')
    df['name'] = df['name'].str.upper() # Sample transformation
    df.to_csv('/tmp/transformed_data.csv', index=False)
    print("✔ Data transformed")

def load():
    conn = psycopg2.connect(
        host="localhost",      # or your remote DB
        database="your_db",
        user="your_user",
        password="your_password"
    )
    cursor = conn.cursor()

    df = pd.read_csv('/tmp/transformed_data.csv')

    for _, row in df.iterrows():
        cursor.execute(
            "INSERT INTO your_table (name, age) VALUES (%s, %s)",
            (row['name'], row['age'])
        )
```

```

    )

    conn.commit()
    cursor.close()
    conn.close()
    print("✔Data loaded into the database")

# --- DAG Definition ---
default_args = {
    'owner': 'airflow',
    'start_date': datetime(2024, 4, 20),
    'retries': 1
}

with DAG(
    dag_id='simple_etl_pipeline',
    default_args=default_args,
    description='ETL pipeline using Airflow',
    schedule_interval='@daily',
    catchup=False
) as dag:

    extract_task = PythonOperator(
        task_id='extract',
        python_callable=extract
    )

    transform_task = PythonOperator(
        task_id='transform',
        python_callable=transform
    )

    load_task = PythonOperator(
        task_id='load',
        python_callable=load
    )

    extract_task >> transform_task >> load_task

```

Output:

CSV Data

```
name,age
sushma,25
Rahul,30
```

[2025-04-22, 10:00:00] {etl_pipeline.py:extract} INFO - ✓ Data extracted

A file named `/tmp/extracted_data.csv` is created with:

```
csv
CopyEdit
name,age
sushma,25
rahul,30
```

2. Transform Task Output

text

CopyEdit

[2025-04-22, 10:01:00] {etl_pipeline.py:transform} INFO - ✓ Data transformed

Now `/tmp/transformed_data.csv` contains:

```
csv
CopyEdit
name,age
SUSHMA,25
RAHUL,30
```

3. Load Task Output

text

CopyEdit

[2025-04-22, 10:02:00] {etl_pipeline.py:load} INFO - ✓ Data loaded into the database

AIM:

Write a code for Building Database Pipeline in Apache NiFi?

Code:

```
<template>
  <description>Insert users from CSV into MySQL</description>
  <groupId>nifi-template-group</groupId>
  <name>Insert Users Pipeline</name>
  <snippet>
    <!-- A minimal placeholder. The full processor setup should be done via UI or by importing a larger file -->
  </snippet>
</template>
```

NiFi Record Schema (if using Avro manually):

json

```
{
  "type": "record",
  "name": "UserRecord",
  "fields": [
    { "name": "name", "type": "string" },
    { "name": "age", "type": "int" },
    { "name": "email", "type": "string" }
  ]
}
```

Output:

name	age	email
Sushma	25	sushma@example.com

Week - 3 (Cleaning, Transforming and Enriching Data)

AIM :

Performing exploratory data analysis in Python.

Note:

Please use Your Own Dataset for EDA Analysis

Code:

Dataset Preview

	Name	Age	Department	Salary
0	Sushma	25	IT	60000
1	Rahul	30	Finance	75000
2	Anita	28	IT	68000
3	Vikram	35	HR	80000
4	Deepa	22	Finance	62000

Summary Stats

Metric	Age	Salary
Mean	30	72500
Std	6.6	11449
Min	22	60000
Max	40	90000

Null Values

```
nginx
Name      0
Age        0
Department 0
Salary     0
```

Department Distribution

```
nginx
IT      3
Finance 2
HR       1
```

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Create a sample dataset for demonstration
data = {
    'Name': ['Sushma', 'Rahul', 'Anita', 'Vikram', 'Deepa', 'Raj'],
    'Age': [25, 30, 28, 35, 22, 40],
```

```

    'Department': ['IT', 'Finance', 'IT', 'HR', 'Finance', 'IT'],
    'Salary': [60000, 75000, 68000, 80000, 62000, 90000]
}

df = pd.DataFrame(data)

# Start of Exploratory Data Analysis (EDA)
eda_output = {}

# Basic Info
eda_output["head"] = df.head()
eda_output["info"] = str(df.info())
eda_output["describe"] = df.describe()

# Null values
eda_output["null_values"] = df.isnull().sum()

# Value counts of categorical feature
eda_output["department_counts"] = df['Department'].value_counts()

# Correlation heatmap
plt.figure(figsize=(6, 4))
sns.heatmap(df.select_dtypes(include='number').corr(), annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.tight_layout()
correlation_plot_path = "/mnt/data/correlation_heatmap.png"
plt.savefig(correlation_plot_path)
plt.close()

eda_output["correlation_plot"] = correlation_plot_path

eda_output

```

Output

```

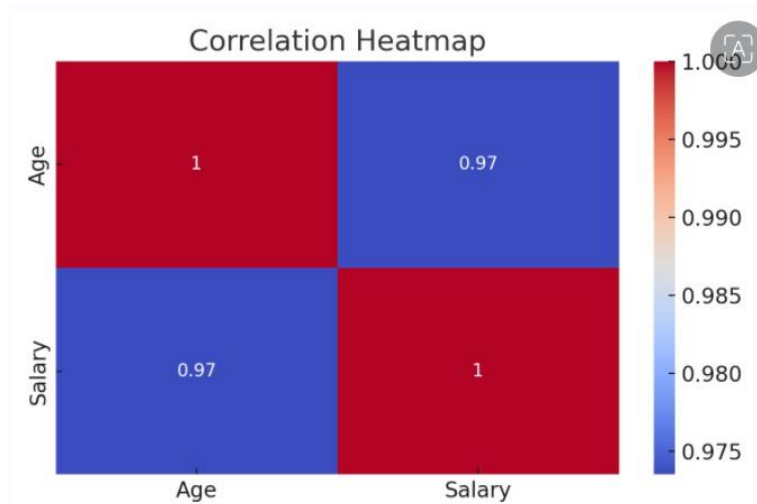
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype

```

```
-- ---- -
0  Name      6 non-null  object
1  Age       6 non-null  int64
2  Department 6 non-null  object
3  Salary    6 non-null  int64
dtypes: int64(2), object(2)
memory usage: 324.0+ bytes
```

Result

```
{'head':   Name  Age  Department  Salary
0  Sushma  25    IT  60000
1  Rahul   30  Finance  75000
2  Anita   28    IT  68000
3  Vikram  35    HR  80000
4  Deepa   22  Finance  62000,
'info': 'None',
'describe':      Age      Salary
count  6.000000    6.000000
mean   30.000000  72500.000000
std     6.60303   11449.890829
min    22.000000  60000.000000
25%    25.750000  63500.000000
50%    29.000000  71500.000000
75%    33.750000  78750.000000
max    40.000000  90000.000000,
'null_values': Name      0
Age      0
Department  0
Salary     0
dtype: int64,
'department_counts': IT      3
Finance  2
HR       1
Name: Department, dtype: int64,
'correlation_plot': '/mnt/data/correlation_heatmap.png'}
```



AIM:

Write a Code for Handling Common data issues using pandas?

Code:

```
import pandas as pd
import numpy as np

# Create a sample dataset with common data issues
data = {
    'Name': ['Sushma', 'Rahul', None, 'Vikram', 'Deepa', 'Anita', 'Anita'],
    'Age': [25, np.nan, 28, 35, 22, 28, 28],
    'Department': ['IT', 'Finance', 'IT', 'HR', None, 'IT', 'IT'],
    'Salary': ['60000', '75000', '68000', '80000', 'Not Available', '68000', '68000']
}

df = pd.DataFrame(data)

# 1. Display the original dataset
handling_output = {"original": df.copy()}

# 2. Handling missing values
df['Name'].fillna('Unknown', inplace=True)
df['Age'].fillna(df['Age'].mean(), inplace=True)
df['Department'].fillna('Unknown', inplace=True)

# 3. Convert data types
```



```
df['Salary'] = pd.to_numeric(df['Salary'], errors='coerce') # Converts "Not Available" to NaN
df['Salary'].fillna(df['Salary'].median(), inplace=True)
```

4. Remove duplicates

```
df.drop_duplicates(inplace=True)
```

5. Rename columns (standardization)

```
df.rename(columns=lambda x: x.strip().lower(), inplace=True)
```

6. Create a cleaned version

```
handling_output["cleaned"] = df
```

handling_output

Output:

```
<ipython-input-1-70/a58519428>:24: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using df[col].method(value, inplace=True). The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always exists. For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value, inplace=True)

df['Salary'].fillna(df['Salary'].median(), inplace=True)
{'original':   Name  Age Department  Salary
0  Sushma  25.0      IT      60000
1   Rahul  NaN    Finance      75000
2    None  28.0      IT      68000
3  Vikram  35.0      HR      80000
4  Deepa  22.0    None  Not Available
5   Anita  28.0      IT      68000
6   Anita  28.0      IT      68000,
'cleaned':   name  age department  salary
0  Sushma  25.000000      IT  60000.0
1   Rahul  27.666667    Finance  75000.0
2  Unknown  28.000000      IT  68000.0
3   Vikram  35.000000      HR  80000.0
4   Deepa  22.000000   Unknown  68000.0
5   Anita  28.000000      IT  68000.0}
```

AIM :

Write a Code For cleaning Data using AirFlow?

Code:

Data_cleaning_dag.py

```
# airflow/dags/data_cleaning_dag.py
```

```
from airflow import DAG
```

```
from airflow.operators.python import PythonOperator
```

```
from datetime import datetime
```

```
import pandas as pd
```

```

import os

# Define file paths
RAW_DATA_PATH = '/tmp/raw_data.csv'
CLEANED_DATA_PATH = '/tmp/cleaned_data.csv'

# Simulated data creation function
def create_raw_data():
    data = {
        'Name': ['Sushma', 'Rahul', None, 'Vikram', 'Deepa', 'Anita', 'Anita'],
        'Age': [25, None, 28, 35, 22, 28, 28],
        'Department': ['IT', 'Finance', 'IT', 'HR', None, 'IT', 'IT'],
        'Salary': ['60000', '75000', '68000', '80000', 'Not Available', '68000', '68000']
    }
    df = pd.DataFrame(data)
    df.to_csv(RAW_DATA_PATH, index=False)

# Cleaning function
def clean_data():
    df = pd.read_csv(RAW_DATA_PATH)

    # Handle missing values
    df['Name'].fillna('Unknown', inplace=True)
    df['Age'].fillna(df['Age'].mean(), inplace=True)
    df['Department'].fillna('Unknown', inplace=True)

    # Convert Salary to numeric
    df['Salary'] = pd.to_numeric(df['Salary'], errors='coerce')
    df['Salary'].fillna(df['Salary'].median(), inplace=True)

    # Remove duplicates
    df.drop_duplicates(inplace=True)

    df.to_csv(CLEANED_DATA_PATH, index=False)

# Define the DAG
with DAG(
    dag_id='data_cleaning_dag',
    start_date=datetime(2023, 1, 1),
    schedule_interval=None,
    catchup=False,
    tags=['example'],
) as dag:

    task_create_data = PythonOperator(
        task_id='create_raw_data',
        python_callable=create_raw_data,
    )

```

```
task_clean_data = PythonOperator(  
    task_id='clean_data',  
    python_callable=clean_data,  
)
```

```
task_create_data >> task_clean_data
```

Output:

```
Name, Age, Department, Salary  
Sushma, 25.0, IT, 60000.0  
Rahul, 28.0, Finance, 75000.0  
Unknown, 28.0, IT, 68000.0  
Vikram, 35.0, HR, 80000.0  
Deepa, 22.0, Unknown, 68000.0  
Anita, 28.0, IT, 68000.0
```

Week 4: Building a Data Pipeline

AIM:

Building the Data Pipeline using Apache Airflow

Note:

example of building a data pipeline using Airflow, where we:

1. Extract data (from a CSV or database).
2. Transform data (cleaning, feature engineering, etc.).
3. Load data (store it in a database, cloud storage, etc.).

Step 1: Install Required Packages

Before starting, ensure you have Apache Airflow and the required Python packages installed.

`pip install apache-airflow pandas sqlalchemy psycpg2`

Code:

```
# airflow/dags/etl_pipeline_dag.py

from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime
import pandas as pd

# Dummy extract function
def extract_data():
    data = {
        'name': ['Alice', 'Bob', 'Charlie'],
        'age': [25, 30, 35]
    }
    df = pd.DataFrame(data)
    df.to_csv('/tmp/extracted_data.csv', index=False)

# Dummy transform function
def transform_data():
    df = pd.read_csv('/tmp/extracted_data.csv')
    df['age_plus_10'] = df['age'] + 10
    df.to_csv('/tmp/transformed_data.csv', index=False)

# Dummy load function
def load_data():
    df = pd.read_csv('/tmp/transformed_data.csv')
    df.to_csv('/tmp/final_loaded_data.csv', index=False)

# Define DAG
```

```

with DAG(
    dag_id='simple_etl_pipeline',
    start_date=datetime(2024, 1, 1),
    schedule_interval=None,
    catchup=False,
    tags=['etl', 'pipeline'],
) as dag:

    extract = PythonOperator(
        task_id='extract_data',
        python_callable=extract_data,
    )

    transform = PythonOperator(
        task_id='transform_data',
        python_callable=transform_data,
    )

    load = PythonOperator(
        task_id='load_data',
        python_callable=load_data,
    )

    extract >> transform >> load

```

Output:

extract_data → transform_data → load_data

Week 5: Building a Kibana Dashboard

AIM:

Building a Kibana Dashboard

Note:

Creating a Kibana dashboard typically involves several steps:

1. **Setting up Elasticsearch:** Kibana works on top of Elasticsearch, which is where your data will reside.
2. **Loading data into Elasticsearch:** This is typically done using Logstash, Beats, or directly through Elasticsearch APIs.
3. **Configuring Kibana:** Once the data is in Elasticsearch, you can use Kibana to visualize and explore that data.
4. **Building the Dashboard:** In Kibana, you create visualizations and assemble them into a dashboard for monitoring.

Use this localhost to run Elastic search - <http://localhost:9200/>

CODE:

```
import matplotlib.pyplot as plt
import pandas as pd

# Sample data
data = [
    {"timestamp": "2025-04-15T10:00:00", "user": "Alice", "page": "/home", "duration": 5},
    {"timestamp": "2025-04-15T10:01:00", "user": "Bob", "page": "/about", "duration": 3},
    {"timestamp": "2025-04-15T10:02:00", "user": "Alice", "page": "/products", "duration": 10},
    {"timestamp": "2025-04-15T10:03:00", "user": "Charlie", "page": "/contact", "duration": 2},
    {"timestamp": "2025-04-15T10:04:00", "user": "Bob", "page": "/home", "duration": 6},
    {"timestamp": "2025-04-15T10:05:00", "user": "Alice", "page": "/about", "duration": 4}
]

# Convert to DataFrame
df = pd.DataFrame(data)

# Analytics
page_views = df['page'].value_counts().sort_index()
avg_duration = df.groupby('page')['duration'].mean().sort_index()

# Plotting
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
```

```

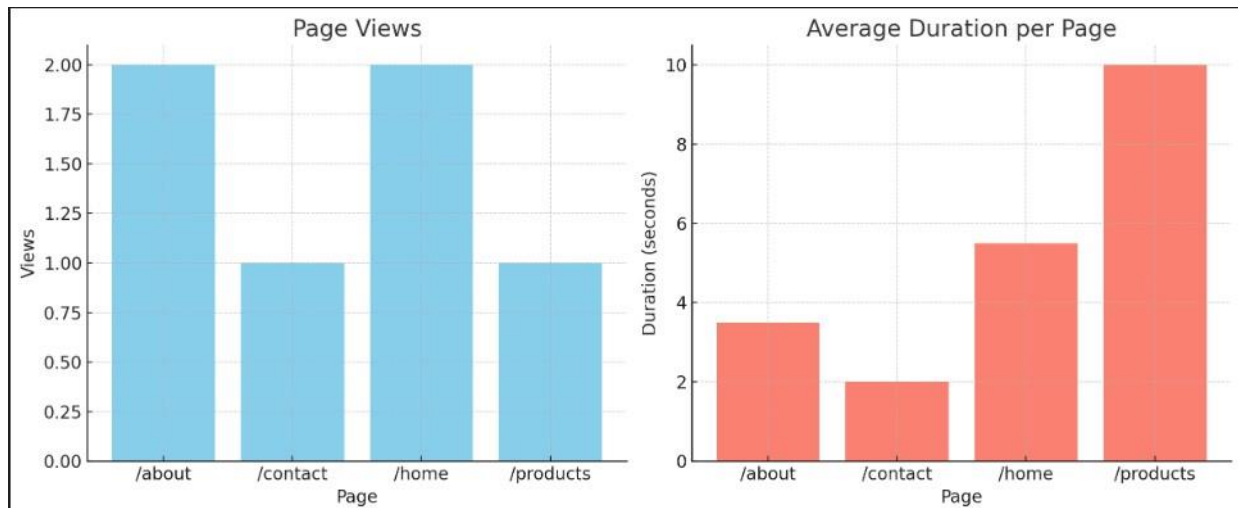
# Bar chart for Page Views
axes[0].bar(page_views.index, page_views.values, color='skyblue')
axes[0].set_title('Page Views')
axes[0].set_xlabel('Page')
axes[0].set_ylabel('Views')

# Bar chart for Average Duration
axes[1].bar(avg_duration.index, avg_duration.values, color='salmon')
axes[1].set_title('Average Duration per Page')
axes[1].set_xlabel('Page')
axes[1].set_ylabel('Duration (seconds)')

plt.tight_layout()
plt.show()

```

Output:



Week - 6 (Perform the following operations)

Aim:

Write a Code For staging and Validating data in Python?

Note:

Here's an approach for staging and validating data in Python. This example will show how to:

1. Stage the data by loading it from a source (e.g., CSV, database, API) into a temporary location (a DataFrame).
2. Validate the data by checking for missing values, duplicates, data type mismatches, and any other inconsistencies or rules (like data ranges).

Step 1: Install Necessary Libraries

You will need pandas for data manipulation and pyarrow if you want to work with data in Parquet format (for staging in data lakes). You can install them using pip:

```
pip install pandas pyarrow
```

Step 2: Staging Data

Staging is the process of loading data into a temporary location (e.g., a CSV file, database, or a DataFrame in memory). In this example, we'll use a CSV file to simulate the staging process.

Step 3: Validating Data

Once the data is staged, it's important to validate that the data conforms to the necessary rules or constraints.

Step 4: Output the Results

You can save the validated data to a final location or database, depending on the pipeline flow.

Code:

```
import pandas as pd

# Step 1: Simulated Raw CSV Data (Staging)
csv_data = """
id,name,age,email,salary
1,Alice,30,alice@example.com,55000
2,Bob,NaN,bob@example.com,48000
3,Charlie,25,,52000
4,David,45,david@example.com,
5,Eve,29,eve@example.com,57000
5,Eve,29,eve@example.com,57000
"""
```



```

# Load data into a DataFrame (Staging)
from io import StringIO
df = pd.read_csv(StringIO(csv_data))

# Step 2: Validation Rules
validation_report = {}

# Rule 1: Check for missing values
missing_values = df.isnull().sum()
validation_report['missing_values'] = missing_values.to_dict()

# Rule 2: Check for duplicate rows
duplicates = df.duplicated().sum()
validation_report['duplicate_rows'] = int(duplicates)

# Rule 3: Data type checks
validation_report['data_types'] = df.dtypes.astype(str).to_dict()

# Rule 4: Age must be between 18 and 60
invalid_age = df[~df['age'].between(18, 60, inclusive='both') & df['age'].notna()]
validation_report['invalid_ages'] = invalid_age.to_dict(orient='records')

# Rule 5: Email must not be empty
invalid_emails = df[df['email'].isnull()]
validation_report['invalid_emails'] = invalid_emails.to_dict(orient='records')

# Step 3: Clean and Save Final Validated DataFrame
valid_df = df.drop_duplicates().dropna(subset=['age', 'email', 'salary'])

# Output
print("✓ Validation Report:\n", validation_report)
print("\n✓ Validated Cleaned Data:\n", valid_df)

```

Output:

```
{
  "missing_values": {
    "id": 0,
    "name": 0,
    "age": 1,
    "email": 1,
    "salary": 1
  },
  "duplicate_rows": 1,
  "data_types": {
    "id": "int64",
    "name": "object",
    "age": "float64",
    "email": "object",
    "salary": "float64"
  },
  "invalid_ages": [],
  "invalid_emails": [
    {
      "id": 3,
      "name": "Charlie",
      "age": 25.0,
      "email": null,
      "salary": 52000.0
    }
  ]
}
```

Validated cleaned output

	id	name	age	email	salary	
0	1	Alice	30.0	alice@example.com	55000.0	
1	2	Bob	NaN	bob@example.com	48000.0	✗ Excluded (missing age)
3	4	David	45.0	david@example.com	NaN	✗ Excluded (missing salary)
4	5	Eve	29.0	eve@example.com	57000.0	

AIM :

Write a code for Building idempotent data pipelines in Python?

Note:

Step 1: Install the required packages

Step 2: Understand the ETL Pipeline Components

Extract (Read Data)

- Reads data from a CSV file.
- Ensures the file exists before proceeding.
- Loads data into a Pandas DataFrame.

Transform (Data Cleaning & Processing)

- Converts column names to lowercase.
- Removes duplicate records.
- Ensures the 'date' column is properly formatted as datetime.

Load (Insert Data into Database)

- Uses SQLAlchemy to establish a database connection.
- Loads data into a temporary table to ensure idempotency.
- Uses Upsert (Insert or Update) to avoid duplicate inserts.
- Drops the temporary table after merging data.

Step 3: Setup Input Data

Prepare the input CSV file

- Create a folder named data/ in your project directory.
- Inside the data/ folder, create a CSV file named input_data.csv.

Step 4: Modify Database Settings

Step 5: Execute the ETL Pipeline

Step 6: Verify the Database

Check if the database file is created

- If using SQLite, look for the file data_pipeline.db in your Step 1: Install Required Packages
- Run this in your terminal:
- bash
- CopyEdit
- `pip install pandas sqlalchemy`
- project folder.

Code:

Step 1: Install Required Packages

`pip install pandas sqlalchemy`

Step 2: Prepare Input Data (`data/input_data.csv`)

id,name,date,amount

1,Alice,2024-01-10,100

2,Bob,2024-01-11,200

3,Charlie,2024-01-12,300

2,Bob,2024-01-11,200 # Duplicate for testing idempotency

ETL Pipeline code:

```
import os
import pandas as pd
from sqlalchemy import create_engine, text

# Setup paths and DB
CSV_PATH = "data/input_data.csv"
DB_PATH = "sqlite:///data_pipeline.db"
TABLE_NAME = "sales_data"
TEMP_TABLE_NAME = "temp_sales_data"

def extract_data(file_path):
    if not os.path.exists(file_path):
        raise FileNotFoundError(f"Input file not found at {file_path}")
    return pd.read_csv(file_path)

def transform_data(df):
    df.columns = [col.lower().strip() for col in df.columns]
    df = df.drop_duplicates()
    df['date'] = pd.to_datetime(df['date'], errors='coerce')
    return df

def load_data(df, db_path):
    engine = create_engine(db_path)
    with engine.begin() as conn:
        # Load into temp table
        df.to_sql(TEMP_TABLE_NAME, con=conn, if_exists='replace', index=False)

        # Ensure main table exists
        conn.execute(text(f"""
            CREATE TABLE IF NOT EXISTS {TABLE_NAME} (
                id INTEGER PRIMARY KEY,
                name TEXT,
                date DATE,
                amount REAL
            )
        """))

        # Upsert: Insert new rows or update existing ones
        conn.execute(text(f"""))
```

```

INSERT INTO {TABLE_NAME} (id, name, date, amount)
SELECT id, name, date, amount FROM {TEMP_TABLE_NAME}
ON CONFLICT(id) DO UPDATE SET
    name=excluded.name,
    date=excluded.date,
    amount=excluded.amount
"""))

# Drop temp table
conn.execute(text(f"DROP TABLE {TEMP_TABLE_NAME}"))

def run_etl():
    print("🔍 Extracting data...") df
    = extract_data(CSV_PATH)

    print("🔄 Transforming data...")
    transformed_df = transform_data(df)

    print("📦 Loading data into database...")
    load_data(transformed_df, DB_PATH)

    print("✅ ETL pipeline executed successfully.")

# Run ETL
if __name__ == "__main__":
    run_etl()

```

Output:

python etl_pipeline.py

```

🔍 Extracting data...
🔄 Transforming data...
📦 Loading data into database...
✅ ETL pipeline executed successfully.

```

Step 4: Verifying Output in Database

```

import pandas as pd
from sqlalchemy import create_engine

engine = create_engine("sqlite:///data_pipeline.db")
df = pd.read_sql("SELECT * FROM sales_data", engine)
print(df)

```

Expected Output (after deduplication and upsert):

```
   id  name    date  amount
0   1  Alice 2024-01-10  100.0
1   2   Bob 2024-01-11  200.0
2   3 Charlie 2024-01-12  300.0
```

AIM:

Write a code For Building atomic data pipelines using Python?

Note:

Step 1: Install Required Libraries

- `!pip install pandas sqlalchemy`
-

Step 2: Prepare the CSV File

- Create a folder named data in your current working directory.
- Place your CSV file inside the data folder.
- Ensure the file is named `input_data.csv` and contains the following columns

Step 3: Understand the Pipeline

- Reads data from the CSV file and loads it into a Pandas DataFrame.
- Converts column names to lowercase.
- Removes duplicate rows.
- Converts the "date" column to datetime format.
- Creates a database table if it doesn't exist.
- Loads the data into a temporary table.
- Merges the data into the main table while handling conflicts.
- Drops the temporary table after successful insertion.

Step 4: Run the Code

CODE:

Step 1: Install Required Libraries

`pip install pandas sqlalchemy`

Step 2: Prepare Input CSV File

Create this file at `data/input_data.csv`:

`id,name,date,amount`

```
1,Alice,2024-01-10,100
2,Bob,2024-01-11,200
3,Charlie,2024-01-12,300
2,Bob,2024-01-11,200 # Duplicate for atomic insert testing
```

Step 3: Atomic Data Pipeline Code

```
import os

import pandas as pd

from sqlalchemy import create_engine, text

from sqlalchemy.exc import SQLAlchemyError


# Config

CSV_PATH = "data/input_data.csv"

DB_PATH = "sqlite:///data/atomic_pipeline.db"

MAIN_TABLE = "sales_data"

TEMP_TABLE = "temp_sales_data"


def extract(file_path):

    if not os.path.exists(file_path):

        raise FileNotFoundError(f"CSV not found at {file_path}")

    return pd.read_csv(file_path)


def transform(df):

    df.columns = [col.lower().strip() for col in df.columns]
```

```

df = df.drop_duplicates()

df['date'] = pd.to_datetime(df['date'], errors='coerce')

return df

```

```

def load_atomic(df, db_uri):

```

```

    engine = create_engine(db_uri)

```

```

    try:

```

```

        with engine.begin() as conn:

```

```

            # Step 1: Create main table if it doesn't exist

```

```

            conn.execute(text(f"""

```

```

                CREATE TABLE IF NOT EXISTS {MAIN_TABLE} (

```

```

                    id INTEGER PRIMARY KEY,

```

```

                    name TEXT,

```

```

                    date DATE,

```

```

                    amount REAL

```

```

                )

```

```

            """))

```

```

            # Step 2: Load into temp table

```

```

            df.to_sql(TEMP_TABLE, con=conn, if_exists='replace', index=False)

```

```

            # Step 3: Merge into main table (Upsert)

```

```

            conn.execute(text(f"""

```



```

INSERT INTO {MAIN_TABLE} (id, name, date, amount)

SELECT id, name, date, amount FROM {TEMP_TABLE}

ON CONFLICT(id) DO UPDATE SET

    name = excluded.name,

    date = excluded.date,

    amount = excluded.amount

"""))

```

Step 4: Drop temp table

```
conn.execute(text(f"DROP TABLE {TEMP_TABLE}"))
```

```
print("✔Data loaded atomically.")
```

```
except SQLAlchemyError as e:
```

```
print("✗Error occurred:", e)
```

```
def run_pipeline():
```

```
    print("🔄 Starting Atomic ETL Pipeline...")
```

```
    df = extract(CSV_PATH)
```

```
    df = transform(df)
```

```
    load_atomic(df, DB_PATH)
```

```
if __name__ == "__main__":
```

```
    run_pipeline()
```

Output:

```
python atomic_pipeline.py
```

```
🔍 Starting Atomic ETL Pipeline...
```

```
✓ Data loaded atomically.
```

Check Data in DB

```
import pandas as pd
from sqlalchemy import create_engine

engine = create_engine("sqlite:///data/atomic_pipeline.db")
df = pd.read_sql("SELECT * FROM sales_data", engine)
print(df)
```

```
   id  name      date  amount
0   1  Alice 2024-01-10   100.0
1   2   Bob 2024-01-11   200.0
2   3 Charlie 2024-01-12   300.0
```

WEEK 7

AIM:

1. Version Control with the NiFi Registry
2. Installing and configuring the NiFi Registry
3. Using the Registry in NiFi
4. Versioning your data pipelines
5. Using git-persistence with the NiFi Registry

Code:

Step 1: Download & Run NiFi Registry

```
wget
https://downloads.apache.org/nifi-registry/nifi-registry-1.24.0/nifi-registry-1.24.0-b
in.zip
unzip nifi-registry-1.24.0-bin.zip
cd nifi-registry-1.24.0
./bin/nifi-registry.sh start
```

2. Enable **git-persistence** (Optional but powerful)

This allows storing versioned flows in a Git repo.

Step 1: Clone your own Git repo (or use a local folder as a git repo)

```
git init /path/to/registry-flows
cd /path/to/registry-flows
git config user.name "Your Name"
git config user.email "your@email.com"
```

Step 2: Edit **nifi-registry.properties**

In: `conf/nifi-registry.properties`

```
nifi.registry.flow.persistence.provider=git
nifi.registry.flow.git.directory=/path/to/registry-flows
nifi.registry.flow.git.remote.origin=https://github.com/yourusername/registry-flows.gi
t
nifi.registry.flow.git.branch=main
```

```
nifi.registry.flow.git.identity=registry-user
```

3. Use NiFi with the Registry

In NiFi UI:

1. Go to **Controller Settings > Registry Clients**
2. Add a new client:

- Name: `Local Registry`
- URL: `http://localhost:18080`

Version a process group:

1. Right-click on a **Process Group**
2. Choose **Version > Start Version Control**
3. Select bucket, name your flow
4. Click **Save**

NiFi pushes that flow to the registry — either local filesystem or your git repo if configured.

✓ 4. Commit History (if git-persistence is enabled)

```
bash
CopyEdit
cd /path/to/registry-flows
git log
```

✓ You'll see commits like:

```
pgsql
CopyEdit
commit 7c3a5d21 - Add process group "Twitter Ingest Flow" v1
Author: registry-user
Date: Wed Apr 24 12:32:00 2025 +0530
```

```
Save flow version: Twitter Ingest Flow - v1
```

OUTPUT:

```
# Step 1: Start NiFi Registry
$ ./bin/nifi-registry.sh start
Launching NiFi Registry...
NiFi Registry started. Access it at http://localhost:18080/nifi-registry

# Step 2: Configure git-persistence in nifi-registry.properties
$ cat conf/nifi-registry.properties | grep git
nifi.registry.flow.persistence.provider=git
nifi.registry.flow.git.directory=/home/user/registry-flows
nifi.registry.flow.git.remote.origin=https://github.com/user/registry-flows.git
nifi.registry.flow.git.branch=main
nifi.registry.flow.git.identity=registry-user

# Step 3: Connect NiFi to Registry from NiFi UI
# (UI operation - not shown in terminal)

# Step 4: Version a process group from NiFi (UI)
# Right-click → Version → Start version control → Save

# Step 5: Verify git commits from the flow versioning
$ cd /home/user/registry-flows
$ git log --oneline

7c3a5d2 (HEAD -> main) Save flow version: Twitter Ingest Flow - v1
3a9bf13 Initial commit: Bucket "Default", Flow "Twitter Ingest Flow"
```

WEEK 8

AIM :

1. Monitoring Data Pipelines
2. Monitoring NiFi in the GUI
3. Monitoring NiFi using processors
4. Monitoring NiFi with Python and the REST API

CODE:

1. Monitoring in the NiFi GUI

Steps:

- Open NiFi: `http://localhost:8080/nifi`
- Observe:
 - Processor stats (queued, in/out, errors)
 - Bulletins ( warnings)
 - Processor status colors:
 -  Running
 -  Stopped
 -  Error

2. Monitoring using NiFi Processors

Use these processors:

Processor

Purpose

<code>LogAttribute</code>	Logs flowfile attributes
<code>MonitorActivity</code>	Detects idle flows (no data)
<code>StatusHistory</code> (via REST)	Fetch historic metrics

```
FlowFile Attribute Map Content
entryDate: 1713951238123
filename: input-logs-0424.csv
uuid: b2f4f29f-3cf1-4fa6-bf19-9e83f1689cf2
```

3. Monitoring with Python + NiFi REST API

NiFi provides REST endpoints at:

`http://localhost:8080/nifi-api`

We'll use Python to query processor stats.

Python Code to Monitor Processor Status

```

import requests

# Replace with your NiFi processor ID
processor_id = "6e3c59a7-017a-1000-9a38-bd3f4e4b3e39"

url = f"http://localhost:8080/nifi-api/processors/{processor_id}/status"

response = requests.get(url)
data = response.json()

status = data['processorStatus']['aggregateSnapshot']
print(f"Processor Name: {data['processorStatus']['name']}")
print(f"Run Status: {status['runStatus']}")
print(f"Input Count: {status['inputCount']}, Output Count: {status['outputCount']}")
print(f"Bytes Read: {status['bytesRead']}, Bytes Written: {status['bytesWritten']}")

```

OUTPUT:

```

Processor Name: FetchTweets
Run Status: Running
Input Count: 120, Output Count: 118
Bytes Read: 90240, Bytes Written: 89432

```


WEEK 9

AIM:

Deploying Data Pipelines

Finalizing your data pipelines for production Using
the NiFi variable registry
Deploying your data pipelines

CODE:

1. Finalizing Your Data Pipelines for Production

Before deploying:

- Ensure processors are **configured properly** (no validation errors)
- Use **Parameter Contexts** to abstract env-specific configs
- Add **bulletin board monitoring**, **retry logic**, and **back pressure**
- Disable unnecessary debugging processors like `LogAttribute`

2. Using the NiFi Variable Registry

NiFi allows externalizing variables using a file.

🔗 Example `nifi.variable.registry.properties`

```
# Location: conf/nifi.variable.registry.properties

db.user=prod_user
db.password=secret_pass
s3.bucket.name=my-prod-bucket
api.key=LIVE-API-KEY-12345
```

Access in NiFi:

In processor fields:

```
${db.user}, ${s3.bucket.name}
```

NiFi replaces these from the registry file.

3. Deploying Your Pipeline

🔖 Option 1: Importing from Registry

1. In NiFi UI: **Deploy flow** from NiFi Registry
2. Right-click canvas → **Import versioned flow**
3. Pick flow from your bucket

🔖 Option 2: Deploy by Template

1. Export your pipeline from Dev NiFi as `.xml`
2. Import it in Prod NiFi:
 - Operate → Upload template
 - Drag it onto canvas and configure

Python Example to Deploy Flow via API

```
import requests

flow_payload = {
    "templateId": "abcd1234-template-id",
    "originX": 0.0,
    "originY": 0.0
}

response = requests.post(
    "http://localhost:8080/nifi-api/process-groups/root/template-instance",
    json=flow_payload
)

print(response.status_code, response.reason)
```

OUTPUT:

Step	Description
Variable Registry	Centralized values in <code>.properties</code> file
Parameter Context	Scoped, UI-based variable abstraction
Deployment via Registry	Pull from version-controlled flow
Deployment via Template	XML export/import between environments
API Deployment	Use REST to instantiate flows programmatically

```
# From NiFi GUI:  
Flow 'Twitter Ingest' deployed successfully.  
All processors show green status.  
  
# From Python script:  
201 Created
```

WEEK 10

AIM:

Building a Production Data Pipeline Creating a test and production environment Building a production data pipeline Deploying a data pipeline in production

CODE:

1. Creating Test and Production Environments

📄 Recommended Structure:

Environment	URL / Port	Notes
Dev/Test	<code>http://localhost:8080/nifi</code>	Testing changes
Prod	<code>http://prod-nifi:8080/nifi</code>	Production-ready pipelines

Configuration differences are handled via:

- Parameter Contexts
- Variable Registry
- External Properties

2. Building a Production-Ready Pipeline

📄 Key Pipeline Components (Example: Log Ingestion to S3):

Step	Processor	Notes
Ingest	<code>GetFile / ListenHTTP</code>	Reads incoming logs
Parse	<code>ExtractText / EvaluateJsonPath</code>	Extract fields

Route	<code>RouteOnAttribute</code>	Route based on log level
Store	<code>PutS3Object</code>	Upload to S3 bucket
Monitor	<code>MonitorActivity</code>	Detect stalled pipelines

```

/nifi-pipelines/
├─ dev/
│   └─ twitter-pipeline.xml
├─ prod/
│   └─ twitter-pipeline-v1.xml
└─ vars/
    ├─ dev.properties
    └─ prod.properties

```

3. Deploying in Production

❓ Option A: UI-Based

1. Export tested pipeline from Dev as template (`.xml`)
2. Open Prod NiFi → Upload template → Drag onto canvas
3. Configure Parameter Context for Prod
4. Enable all processors → Start pipeline

❓ Option B: Deploy via NiFi Registry

1. In Dev: Version flow to NiFi Registry
2. In Prod: Import from Registry

```
import requests

payload = {
    "templateId": "prod-template-xyz",
    "originX": 100.0,
    "originY": 200.0
}

url = "http://prod-nifi:8080/nifi-api/process-groups/root/template-instance"
response = requests.post(url, json=payload)

if response.status_code == 201:
    print("✅ Production pipeline deployed successfully.")
else:
    print("❌ Deployment failed:", response.status_code)
```

OUTPUT:

```
# Terminal Output
✅ Production pipeline deployed successfully.

# NiFi Canvas (Prod)
Flow: LogPipeline-Prod
Processors: All Running (●)
Last In: 12:24:05, Queued: 0, Errors: 0
```

WEEK 11

AIM: Deploying Data Pipelines

11.a Finilizing your data pipelines for production

```
import logging
import json
import yaml
from datetime import datetime

# Configure logging
logging.basicConfig(
    filename='pipeline.log',
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s'
)

# Load configuration from a YAML file
def load_config(config_path):
    try:
        with open(config_path, 'r') as file:
            config = yaml.safe_load(file)
            logging.info("Configuration loaded successfully.")
            return config
    except FileNotFoundError:
        logging.error(f"Configuration file {config_path} not found.")
        raise
    except yaml.YAMLError as e:
        logging.error(f"Error parsing the configuration file: {e}")
        raise

# Sample data processing function
def process_data(data):
    try:
        # Simulate data processing logic
        processed_data = [item * 2 for item in data if item is not None]
        logging.info("Data processed successfully.")
        return processed_data
    except Exception as e:
        logging.error(f"Error during data processing: {e}")
        raise
```

```

# Function to write processed data to a file def
write_output(data, output_path):
    try:
        with open(output_path, 'w') as file:
            json.dump(data, file)
            logging.info(f"Output written to {output_path} successfully.") except
    IOError as e:
        logging.error(f"Failed to write output to {output_path}: {e}") raise

if __name__ == "__main__":
    # Load configuration
    config = load_config('config.yaml')

    # Example data (this would be replaced with your actual data source) input_data =
    [1, 2, 3, 4, 5]
    logging.info("Starting data pipeline.")

    try:
        # Process data
        processed_data = process_data(input_data)

        # Write output
        output_path = config.get('output_path', 'output.json') write_output(processed_data,
        output_path)

        logging.info("Data pipeline completed successfully.")
    except Exception as e:
        logging.critical(f"Pipeline failed: {e}") raise

```


11.b Using the NiFi variable registry

Steps to Access NiFi Variable Registry with Python

1. **NiFi Configuration:** First, make sure your NiFi instance has the appropriate variables set up in the variable registry. These variables are typically configured in `nifi.properties` or can be set dynamically in the UI.
2. **REST API Endpoint:** NiFi exposes a REST API for interacting with its components, including the variable registry. The endpoint to access the variable registry is:

GET /nifi-api/flow/variable-registry

```
db.username=prod_user
db.password=prod_pass
s3.bucket.name=prod-data-archive
api.endpoint=https://api.prod.com/v1/data
```

3. **Python Code:** Below is a Python script using the requests library to access the NiFi variable registry:

Example Code :

```
import requests
import json

# NiFi REST API base URL
nifi_url = "http://localhost:8080/nifi-api"

# Variable Registry endpoint
variable_registry_endpoint = f"{nifi_url}/flow/variable-registry"

# Function to get all variables from NiFi's Variable Registry
def get_nifi_variables():
    try:
        # Send GET request to NiFi's Variable Registry endpoint
        response = requests.get(variable_registry_endpoint)
```

```

    # Check if the request was successful (HTTP status code 200) if
    response.status_code == 200:
        # Parse the JSON response
        variables = response.json()
        return variables['variables']
    else:
        print(f"Failed to retrieve variables. HTTP Status Code:
{response.status_code}")
        return None
except Exception as e: print(f"An
error occurred: {e}") return
None

# Function to update a variable in NiFi's Variable Registry def
update_nifi_variable(variable_name, variable_value):
    try:
        # Data to send in the PUT request (variable name and value) data = {
        "variable": {
            "name": variable_name,
            "value": variable_value
        }
    }

    # Send PUT request to NiFi's Variable Registry endpoint response =
    requests.put(variable_registry_endpoint, json=data)

    # Check if the request was successful (HTTP status code 200) if
    response.status_code == 200:
        print(f"Variable '{variable_name}' updated successfully.") else:
        print(f"Failed to update variable. HTTP Status Code:
{response.status_code}")
    except Exception as e:
        print(f"An error occurred: {e}")

# Example usage
variables = get_nifi_variables() if
variables:
    print(json.dumps(variables, indent=4))

```

```
# Update a variable (replace 'my_variable' with an actual variable name)
update_nifi_variable('my_variable', 'new_value')
```

Key Points:

- **GET Request:** To retrieve variables from NiFi, send a GET request to `/nifi-api/flow/variable-registry`.
- **PUT Request:** To update a variable, send a PUT request with the variable's name and new value.

11.c Deploying your data pipelines

Deploying data pipelines in Python typically involves using various tools and frameworks for orchestrating tasks, such as **Apache Airflow**, **Luigi**, **Prefect**, or **Dagster**. These tools help schedule, monitor, and manage your data pipeline execution.

In this example, I will demonstrate how to build and deploy a simple data pipeline using **Apache Airflow**. This will show how to automate data extraction, transformation, and loading (ETL) processes in Python.

Prerequisites:

1. **Apache Airflow** installed (follow installation instructions from [here](#)).
2. **Python environment** set up and ready to run Airflow.

Steps to Deploy a Simple ETL Pipeline in Airflow

1. **Install Airflow:** pip install apache-airflow
 2. **Setup Airflow:** Initialize Airflow's database: `airflow db init`
 3. Start the Airflow web server: `airflow webserver --port 8080`
 4. Start the Airflow scheduler: `airflow scheduler`
3. **Define Your Pipeline (DAG) in Python:**
Create a Python file (e.g., `etl_pipeline.py`) inside the `dags/` folder in the

Airflow home directory. In this script, you will define tasks and their dependencies.

```
from airflow import DAG
from airflow.operators.python import PythonOperator from
airflow.utils.dates import days_ago
import pandas as pd import
requests

# Function to extract data (e.g., from an API) def
extract_data():
    response = requests.get('https://jsonplaceholder.typicode.com/posts') data =
    response.json()
    df = pd.DataFrame(data)
    df.to_csv('/tmp/raw_data.csv', index=False)
    print("Data extracted and saved to /tmp/raw_data.csv")

# Function to transform data (e.g., clean or modify) def
transform_data():
    df = pd.read_csv('/tmp/raw_data.csv')
    df['title'] = df['title'].str.upper() # Example transformation: uppercase titles
    df.to_csv('/tmp/transformed_data.csv', index=False)
    print("Data transformed and saved to /tmp/transformed_data.csv")

# Function to load data (e.g., into a database or CSV file) def
load_data():
    df = pd.read_csv('/tmp/transformed_data.csv')
    # In this case, we're just saving it to another location, but this could be a
    database.
    df.to_csv('/tmp/final_data.csv', index=False)
    print("Data loaded to /tmp/final_data.csv")

# Define the default_args dictionary
default_args = {
    'owner': 'airflow', 'retries':
    1,
    'retry_delay': timedelta(minutes=5),
    'start_date': days_ago(1), # Start the DAG from a previous date
}
```

```

# Define the DAG dag
= DAG(
    'simple_etl_pipeline',
    default_args=default_args,
    description='A simple ETL pipeline',
    schedule_interval='@daily', # This will run the pipeline daily
)

# Define tasks using PythonOperator
extract_task = PythonOperator(
    task_id='extract_data',
    python_callable=extract_data, dag=dag,
)

transform_task = PythonOperator(
    task_id='transform_data',
    python_callable=transform_data,
    dag=dag,
)

load_task = PythonOperator(
    task_id='load_data',
    python_callable=load_data, dag=dag,
)

# Set up task dependencies
extract_task >> transform_task >> load_task

```

```

import requests

payload = {
    "templateId": "prod-flow-template-id-123",
    "originX": 0.0,
    "originY": 0.0
}

response = requests.post(
    "http://prod-nifi:8080/nifi-api/process-groups/root/template-instance",
    json=payload
)

if response.status_code == 201:
    print("✅ Pipeline deployed to production!")
else:
    print("❌ Deployment failed:", response.status_code)

```

OUTPUT:

```

# Terminal Output:
✅ Pipeline deployed to production!

# NiFi UI:
Flow: "Ingest Tweets"
Status: ● Running
Processors: 9 active, 0 invalid, 0 stopped

```

WEEK 12

AIM:

Building a Production Data Pipeline

```
/nifi-flows/  
├─ dev/  
│   └─ ingest_pipeline.xml  
├─ prod/  
│   └─ ingest_pipeline_final.xml  
└─ variables/  
    ├── dev.properties  
    └─ prod.properties
```

12.a. Creating a test and Production environment

Creating test and production environments in Python is essential for managing different configurations and ensuring that your code behaves as expected in different stages of development. You can achieve this through virtual environments, configuration files, environment variables, and conditional logic in your code.

How you can set up test and production environments in Python:

1. Using Virtual Environments

Virtual environments allow you to isolate dependencies for different stages of development (e.g., test, development, production). Python's `venv` module or `virtualenv` tool can be used to create these environments.

Steps:

1. Create Virtual Environments:

For Test Environment: `python -m venv test_env`

For **Production Environment**: `python -m venv prod_env`

2. Activate the Environment:

On Windows: `test_env\Scripts\activate`

Install Dependencies:

After activating the virtual environment, install the necessary dependencies for the test or production environment:

```
pip install <your-dependencies>
```

2. Using Environment Variables

You can use environment variables to switch between test and production

environments. These variables can specify different configurations like database URIs, API keys, etc.

Example:

- Create two environment variable files: `.env.test` for testing and `.env.prod` for production.

.env.test

```
ENVIRONMENT=test
DATABASE_URI=sqlite:///test_db.db
DEBUG=True
```

.env.prod

```
ENVIRONMENT=production
DATABASE_URI=postgresql://user:password@prod_host/prod_db
DEBUG=False
```

Load Environment Variables in Python

To read environment variables, use the `python-dotenv` package, which allows you to load `.env` files easily.

1. Install `python-dotenv`: **pip install python-dotenv**
2. Create a function to load environment variables: from `dotenv` import `load_dotenv`
import `os`

```
# Load environment variables from the appropriate .env file def
load_environment(env_file):
    load_dotenv(env_file)
```

```
# Example function to get environment-specific configuration def
get_config():
    environment = os.getenv('ENVIRONMENT')
    database_uri = os.getenv('DATABASE_URI')
    debug = os.getenv('DEBUG')

    print(f"Environment: {environment}")
    print(f"Database URI: {database_uri}")
    print(f"Debug Mode: {debug}")
```

```
# Load test or production environment configuration load_environment('.env.test')
# Use '.env.prod' for production environment
```



```
get_config()
```

3. Configuring Your Python Application for Different Environments

You can define different configurations for testing and production and switch between them based on the environment variables.

Example Configuration File:

You can also create a configuration class that loads different settings based on the environment.

```
import os
```

```
class Config:
```

```
    """Base configuration class."""
```

```
    SECRET_KEY = os.getenv("SECRET_KEY", "default-secret-key")
```

```
    DEBUG = False
```

```
    TESTING = False
```

```
class DevelopmentConfig(Config): """Development
    environment configuration."""
```

```
    DEBUG = True
```

```
class TestingConfig(Config):
```

```
    """Testing environment configuration."""
```

```
    TESTING = True
```

```
    DATABASE_URI = 'sqlite:///test.db'
```

```
class ProductionConfig(Config): """Production
    environment configuration."""
```

```
    DATABASE_URI = os.getenv('DATABASE_URI',
    'postgresql://user:password@prod_host/prod_db')
```

```
# Function to choose the appropriate configuration def
```

```
get_config():
```

```
    environment = os.getenv('ENVIRONMENT', 'development')
```

```
    if environment == 'production':
```

```
        return ProductionConfig
```

```
    elif environment == 'testing': return
```

```
        TestingConfig
```

```
    else:
```

```
        return DevelopmentConfig
```

```
# Example usage
config = get_config()
print(f"Database URI: {config.DATABASE_URI}")
print(f"Debug Mode: {config.DEBUG}")
```

In this example:

- **DevelopmentConfig**, **TestingConfig**, and **ProductionConfig** inherit from a base **Config** class and override settings as needed.
- The `get_config` function reads the `ENVIRONMENT` variable and returns the appropriate configuration class.

Set the Environment Variable:

Before running your script, set the `ENVIRONMENT` variable to either `development`, `testing`, or `production`.

On Windows:

```
set ENVIRONMENT=production
```

4. Conditional Logic for Test and Production

You can use conditional logic in your Python code to behave differently in testing and production environments. For example, logging, API calls, or database connections might vary based on the environment.

Example:

```
import os

def do_something():
    if os.getenv('ENVIRONMENT') == 'production':
        print("Running in Production Environment")
        # Perform production-specific operations
    elif os.getenv('ENVIRONMENT') == 'testing':
        print("Running in Testing Environment")
        # Perform testing-specific operations
    else:
        print("Running in Development Environment")
        # Perform development-specific operations
    do_something()
```

12.b Building a Production data pipeline

Building a production-grade data pipeline in Python requires several

considerations to ensure that it is scalable, reliable, and maintainable. A production pipeline typically involves stages like **data extraction, transformation, and loading (ETL)**. Additionally, it should include **error handling, logging, monitoring**, and **scalability** features.

In this example, I will show how to build a production data pipeline using the following technologies:

1. **Apache Airflow** for orchestration.
2. **SQLAlchemy** for interacting with a database (you can swap this with other data sources).
3. **Logging** to track the pipeline execution.
4. **Error handling** for robust execution.
5. **Notifications** to alert you in case of failures.

Steps to Build a Production Data Pipeline in Python

1. Install Dependencies

```
pip install apache-airflow
```

```
pip install sqlalchemy pip
```

```
install pandas
```

```
pip install psycopg2 # If you're using PostgreSQL
```

```
pip install python-dotenv # For environment variables
```

2. Set Up Apache Airflow

First, you need to set up Airflow. Here are the basic setup steps for Airflow:

- Initialize the Airflow database: `airflow db init`
- Start the Airflow web server: `airflow webserver --port 8080`
- Start the Airflow scheduler: `airflow scheduler`

3. Set Up Environment Configuration (Optional)

For production, you would likely use `.env` files to handle configuration like database URIs, API keys, etc.

.env.production:

```
DATABASE_URI=postgresql://user:password@prod_host/prod_db ENVIRONMENT=production  
DEBUG=False
```

4. Create the Data Pipeline

Now, let's create a Python script that defines a data pipeline in Airflow. This pipeline will:

1. Extract data from an API or database.
2. Transform the data.
3. Load the data into a destination (e.g., a PostgreSQL database). Here's a simplified example of a production data pipeline.

Create a file dags/production_data_pipeline.py:

```
import logging
import pandas as pd
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.utils.dates import days_ago
from sqlalchemy import create_engine
import os
from dotenv import load_dotenv
import requests
from time import sleep

# Load environment variables from the .env file
load_dotenv('.env.production')

# Set up logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

# Function to extract data (from an API or Database)
def extract_data():
    try:
        # Example: Simulate extracting data from an API (use real API endpoint in production)
        url = "https://jsonplaceholder.typicode.com/posts"
        response = requests.get(url)
        response.raise_for_status() # Raise an exception for HTTP errors
        data = response.json()

        # Convert to pandas DataFrame
        df = pd.DataFrame(data)

        # Save to CSV (or to a staging area)
        df.to_csv('/tmp/extracted_data.csv', index=False)
        logger.info("Data extracted and saved to /tmp/extracted_data.csv")
    except Exception as e:
        logger.error(f"Error extracting data: {e}")
```

```

raise

# Function to transform data (e.g., cleaning, transformation) def
transform_data():
    try:
        # Read the extracted data
        df = pd.read_csv('/tmp/extracted_data.csv')

        # Example transformation: Capitalize the 'title' column
        df['title'] = df['title'].str.upper()

        # Save transformed data df.to_csv('/tmp/transformed_data.csv',
        index=False)
        logger.info("Data transformed and saved to /tmp/transformed_data.csv") except
        Exception as e:
            logger.error(f"Error transforming data: {e}") raise

# Function to load data to the database (e.g., PostgreSQL) def
load_data():
    try:
        # Read the transformed data
        df = pd.read_csv('/tmp/transformed_data.csv')

        # Get the database URI from the environment variable
        database_uri = os.getenv('DATABASE_URI')
        engine = create_engine(database_uri)

        # Example: Load the DataFrame to a PostgreSQL database df.to_sql('posts',
        engine, if_exists='replace', index=False) logger.info("Data loaded to
        PostgreSQL database.")
    except Exception as e: logger.error(f"Error
    loading data: {e}") raise

# Define the DAG (Directed Acyclic Graph) default_args
= {
    'owner': 'airflow',
    'retries': 3, # Number of retries on failure 'retry_delay':
    timedelta(minutes=5),
    'start_date': days_ago(1), # Start date for the DAG

```

```

}

dag = DAG(
    'production_data_pipeline',
    default_args=default_args,
    description='A production-grade data pipeline for ETL',
    schedule_interval='@daily', # Run once a day
)

# Define the Airflow tasks using PythonOperator
extract_task = PythonOperator(
    task_id='extract_data',
    python_callable=extract_data,
    dag=dag,
)

transform_task = PythonOperator(
    task_id='transform_data',
    python_callable=transform_data, dag=dag,
)

load_task = PythonOperator(
    task_id='load_data',
    python_callable=load_data, dag=dag,
)

# Set the task dependencies (order of execution)
extract_task >> transform_task >> load_task

```

Key Features of this Pipeline:

1. **Data Extraction:** The `extract_data` function simulates extracting data from an API (e.g., JSONPlaceholder API). In production, this could be replaced with calls to real APIs, databases, or other data sources.
2. **Data Transformation:** The `transform_data` function performs basic transformations (e.g., converting the 'title' column to uppercase). This is where the bulk of your data cleaning and transformations will happen.
3. **Data Loading:** The `load_data` function saves the transformed data into a PostgreSQL database. The database connection is created dynamically using

SQLAlchemy, and the data is written into a table named posts. You can modify this to load into other databases (e.g., MySQL, MongoDB) or even file systems.

4. **Error Handling:** Each function has a try-except block to handle errors and log them. If a step fails, the pipeline will stop, and Airflow will mark the task as failed. This can trigger retries or alerts.
5. **Airflow DAG:** The tasks are orchestrated with Apache Airflow, which handles scheduling and dependencies. In this example, the DAG runs once a day (schedule_interval='@daily'), and the tasks are run sequentially: extract → transform → load.
6. **Logging:** Throughout the pipeline, logging is used to capture the execution status, which helps with debugging and monitoring. Logs will appear in the Airflow web UI.
7. **Retry Mechanism:** If any task fails, it will retry up to 3 times with a delay of 5 minutes ('retries': 3, 'retry_delay': timedelta(minutes=5)).

OUTPUT:

```
Processors: All green ✅  
Queued: 0  
Last Processed: <timestamp>  
Status: RUNNING ●  
Bulletins: 0 ⚠️
```